Ajaia LLC: AI-Native Full Stack Developer Assignment

AI-Native Workflow Note

AI-Tools used:
GitHub Copilot - For boilerplating, gluing together the parts of the program, debugging, finding security flaws.
DeepSeek web - For brainstorming design, quality, and understanding necessary architecture.

AI decisions I rejected:

The AI tools forgot about a stored-XSS security flaw. I value security in front end applications so I decided to investigate any XSS flaws. There was a flaw where the “PATCH /api/documents/:id“ route accepts “contentHtml” and writes it straight into the database with 0 sanitization before persisting. The tools did make the browser sanitize on render in my public/app.js, but that only helped the current client. Any malicious or compromised user can store raw HTML (like <script>, onerror=..., any unsafe tags etc.) and it becomes persistent data for all the other users and shared-to friends. This is not good so it was patched.

The LLM first wanted to only use Playwright for testing, but I knew of Vitest. I used Vitest+Supertest to test the program’s features, like the main app features, permissions, API requests, and HTTP requests before launching the browser version of the program. I let the LLM use Playwright for most of the browser testing. Together, we were able to fix issues along the way such as bad HTTP request issues.

Multiple style decisions, were rejected:

The LLMs put the delete document button very close to the Save button within the document page. This was a very dangerous design. I changed it so the Delete document button was within a dropdown at the bottom of the page and a feature to ask the user if they actually wanted to delete the file or not.

The LLMs used basic test names like alice, bob, carol. So these were changed.

When adding new features, the agent I used did not follow the style of the project, I changed it so new features matched the style.

How I verified correctness, UX quality, and implementation reliability:

I decided to do multiple tests.
As stated in the previous section, I used Vitest and Playwright to test major features. Vitest was useful in finding errors before the program was in the browser. Playwright tested features within the browser. Within the workflow, all tests that were created were passed without errors.

I tested to make sure the project can actually be loaded. I was able to make it so the program used a Docker file to hold the program and that docker can run it. Before actually running the program with Docker I had to make sure I had Docker installed correctly and updated to make sure there isn't a problem that comes specifically from Docker. After successfully getting Docker to work, I used a basic localhost to host and test. I successfully ran the program.

Using the tests I was able to make sure that users and their content were separated reliably, which was done with ease thanks to a Vitest test.

I tested the main features. I tested many basic design and HTML features before making the program a Docker image. Doing this allowed me to brainstorm what the website should look like. I used Deepseek to come up with designs. I chose a very minimalistic design and the title "Docs with Friends", avoiding messy and cliche indigo+gradient designs. I prompted the LLM to make sure it was very user friendly: readable font, buttons are large and change color when the user hovers over them, not too much text.

While working with AI tools, I made sure I worked piece by piece rather than give everything at once. This is to prevent miscommunication issues that can create unchecked bugs if all directions were given at once. I put a tight token budget on this project (based on what percentage of tokens I have left in my monthly plan) to prevent spending too many tokens on this project. This made me prioritize great prompt engineering and using Deepseek Chat to check if prompts can be misinterpreted and give questions to design choices, while also being very careful with bugs. However the budget prevented me from going further in adding more advanced features when I had time left.